Universidad de Las Palmas de Gran Canaria · 3ºGCID · Big Data

Assignment 01: Basic Matrix Multiplication in Python, Java, and C

Author: Hugo Barreto Suárez | **Submission:** 16 Oct. 2026 | **Repository:** GitHub

Abstract

This report investigates how square matrix dimension (n) and language execution model affect kernel execution time and memory usage for the naive O(n^3) triple-loop dense matrix multiplication algorithm implemented in Python, Java, and C. Using identical 64-bit floating-point inputs generated by a shared 64-bit Linear Congruential Generator (LCG), we benchmarked sizes $n \in \{10, 50, 100, 200, 300, 500, 800, 1000\}$ under a 120-second per-configuration timeout budget with 2 warm-up runs and 5 measured repetitions. Results confirm cubic runtime scaling across all three implementations, separated by large constant factors: C and JIT-compiled Java completed up to $n = 1000$ (approximately 0.60 s and 2.80 s median time, respectively), whereas interpreted CPython reached 11.5 s per multiplication at $n = 500$ and timed out at $n = 800$. These conclusions hold specifically for sequential, unvectorized textbook loops under the tested hardware and runtime settings.

1. Introduction, Objective, and Background

Dense matrix multiplication ($C = A \times B$) is a foundational kernel in data science, numerical linear algebra, and machine learning. For two square $n \times n$ matrices, each entry of the product matrix is defined as $C_{i,j} = \sum_{k=0}^{n-1} A_{i,k} B_{k,j}$. Computing all n^2 entries via the textbook triple nested loop requires n^3 multiplications and n^3 additions, yielding an asymptotic time complexity of $O(n^3)$ and spatial complexity of $O(n^2)$ [1].

Research Question & Hypothesis: *How does matrix size (n) affect the kernel execution time and memory consumption of a basic triple-loop matrix multiplication across Python, Java, and C under controlled conditions?* We hypothesize that all three implementations will follow a slope of approximately 3 on a log-log runtime plot due to the $O(n^3)$ arithmetic workload, but with orders-of-magnitude differences in constant factors driven by CPython bytecode dispatch and object boxing versus Java JIT compilation and C native machine code, as well as cache locality differences during column-wise traversal of matrix B [2].

2. Methodology and Experimental Protocol

2.1 Language Implementations and Storage Layout: To isolate language runtime and memory layout effects without external library calls, the exact same `(i, j, k)` loop order was implemented in all three languages using 64-bit IEEE 754 floating-point arithmetic (`float64`). In each implementation, row references for A and C are hoisted outside the inner loops, and the dot product is accumulated in a local scalar variable (`total` or `sum`) before writing to $C_{i,j}$:

- **C (`src/matmul.c`):** Matrices are allocated as single contiguous 1D row-major blocks via `calloc(n * n, sizeof(double))`, indexed explicitly as `i * n + k` and `k * n + j`.
- **Java (`src/MatMul.java`):** Matrices use standard 2D arrays (`double[n][n]`), representing an array of n object references pointing to n separate 1D `double[]` row arrays on the JVM heap.
- **Python (`src/matmul.py`):** Matrices are stored as nested lists of lists (`[[0.0] * n for _ in range(n)]`), where each element is a pointer to a heap-allocated `PyFloatObject`.

2.2 Controlled Input Generation, Environment, and Correctness Validation

Standard random number generators differ across C, Java, and Python. To guarantee identical logical input matrices without introducing disk I/O overhead, all three programs implement the exact same 64-bit Linear Congruential Generator (LCG): `state = (state * 6364136223846793005 + 1442695040888963407) mod 2^64`. In each step, the upper 53 bits are extracted via unsigned right shift (`state >> 11`, or `state >>> 11` in Java) and multiplied by 2^{-53} (`1.0 / 9007199254740992.0`) to yield identical uniform values in [0, 1). Matrix A uses `seed` and Matrix B uses `seed + 1`.

Before running performance benchmarks, correctness was verified at $n = 16$ (`seed = 42`) using three built-in CLI modes: (1) `identity` (verifying $A \times I = A$ with maximum absolute error 0.0), (2) `zero` (verifying $A \times 0 = 0$ with maximum error 0.0), and (3) `dump` (comparing all 256 elements of $C = A \times B$ against NumPy's reference `np.matmul(A_ref, B_ref)` using `np.allclose` with tolerances `rtol = 1e-12` and `atol = 1e-12`). All three implementations passed with maximum absolute error below 10^{-15}, consistent with IEEE 754 double-precision rounding (machine epsilon $\approx 2.22 \times 10^{-16}$) [3].

Component	C Implementation	Java Implementation	Python Implementation
Source & Compiler / Runtime	`src/matmul.c` GCC 16.2, flags: `-O2 -lpsapi` (Win) / `-lm`	`src/MatMul.java` OpenJDK 17.0.20.1, standard JIT settings	`src/matmul.py` CPython 3.14.6 (64-bit, Anaconda env)
Storage Layout	Contiguous 1D `double*` array (row-major, $8n^2$ bytes)	2D array `double[n][n]` (array of n row pointers)	List of lists `list[list[float]]` (boxed float objects)

Monotonic Timer	Win32 `QueryPerformanceCounter` / POSIX `CLOCK_MONOTONIC`	`System.nanoTime()` converted to seconds (÷ 10^9)	`time.perf_counter_ns()` converted to seconds (÷ 10^9)
Memory Metric & Tool	Peak Working Set (Win32 `GetProcessMemoryInfo`) / `ru_maxrss`	Used JVM Heap: `totalMemory() - freeMemory()`	Peak Working Set / RSS via `psutil.Process().memory_info()`
Hardware Environment	OS: Windows 11 · CPU: Ultra Core 7 155H (16 physical cores / 22 logical processors) · RAM: 15.37 GB · Sequential execution (`threads = 1`)		

2.3 Timing Boundary, Repetition Protocol, and Resource Budgets

Kernel Timing Boundary: We measure strictly the matrix multiplication kernel (`multiply_kernel`). Matrix memory allocation and LCG input generation for A and B occur once outside the repetition loop. For each configuration, **2 unmeasured warm-up runs** are executed first to allow JVM JIT compilation in Java and instruction/data cache warming across all languages. Subsequently, **5 measured repetitions** are recorded (`seed = 12345`). Before each repetition, matrix C is zeroed outside the timer. Immediately after the timer stops, a full checksum ($\sum_{i,j} C_{i,j}$) is computed and printed to standard output to verify deterministic execution and prevent dead-code elimination by the C compiler or Java JIT.

Matrix Size Sequence & Timeout Budget: We evaluated $n \in \{10, 50, 100, 200, 300, 500, 800, 1000\}$. A hard process timeout budget of **120.0 seconds** per subprocess invocation (covering 2 warm-ups + 5 measured runs) was enforced via Python's `subprocess.run(..., timeout=120.0)`. If a language exceeded 120 seconds at size n, the run was logged as `TIMEOUT_EXCEEDED_120.0S` and larger sizes were skipped (`SKIPPED_AFTER_TIMEOUT`).

3. Experimental Results

Across all completed runs, the five repetitions per configuration produced identical checksums across C, Java, and Python, confirming numerical consistency. Figures 1 and 2 illustrate the median kernel execution time (with interquartile range, IQR, error bars) and median memory usage across the tested matrix sizes. Table 2 summarizes the benchmark measurements alongside execution status under the 120-second process budget.

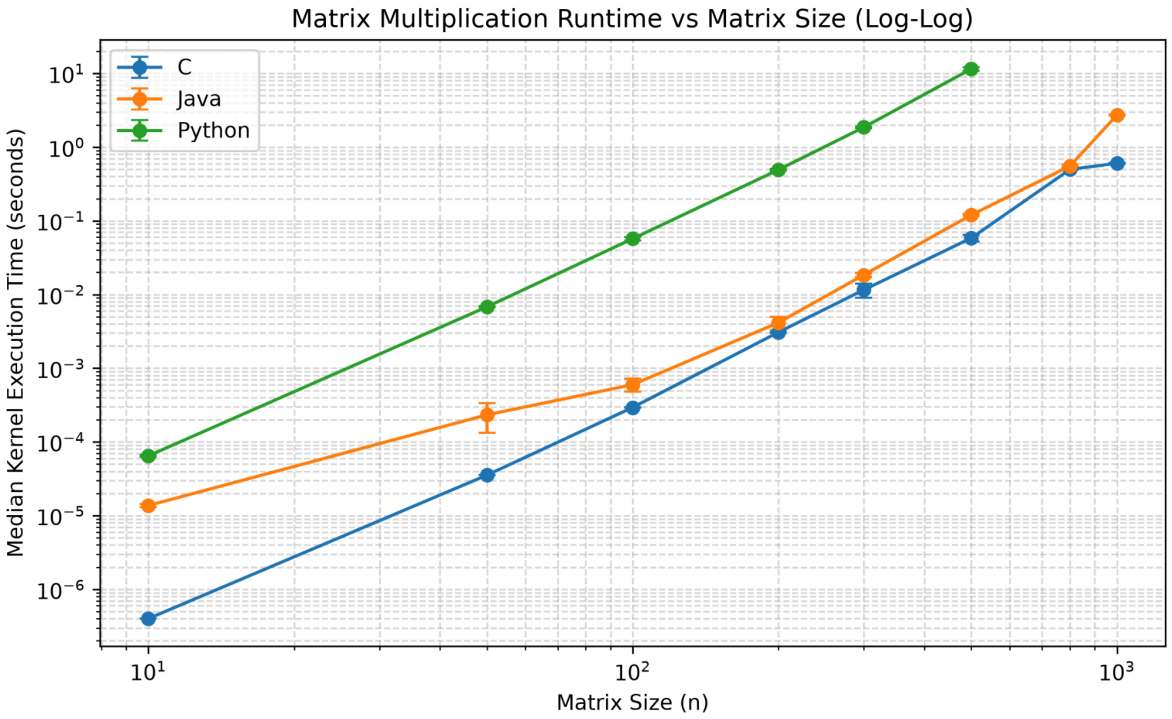


Figure 1: Median kernel execution time (s) vs. matrix size *n* on log-log axes, with IQR error bars over 5 repetitions.

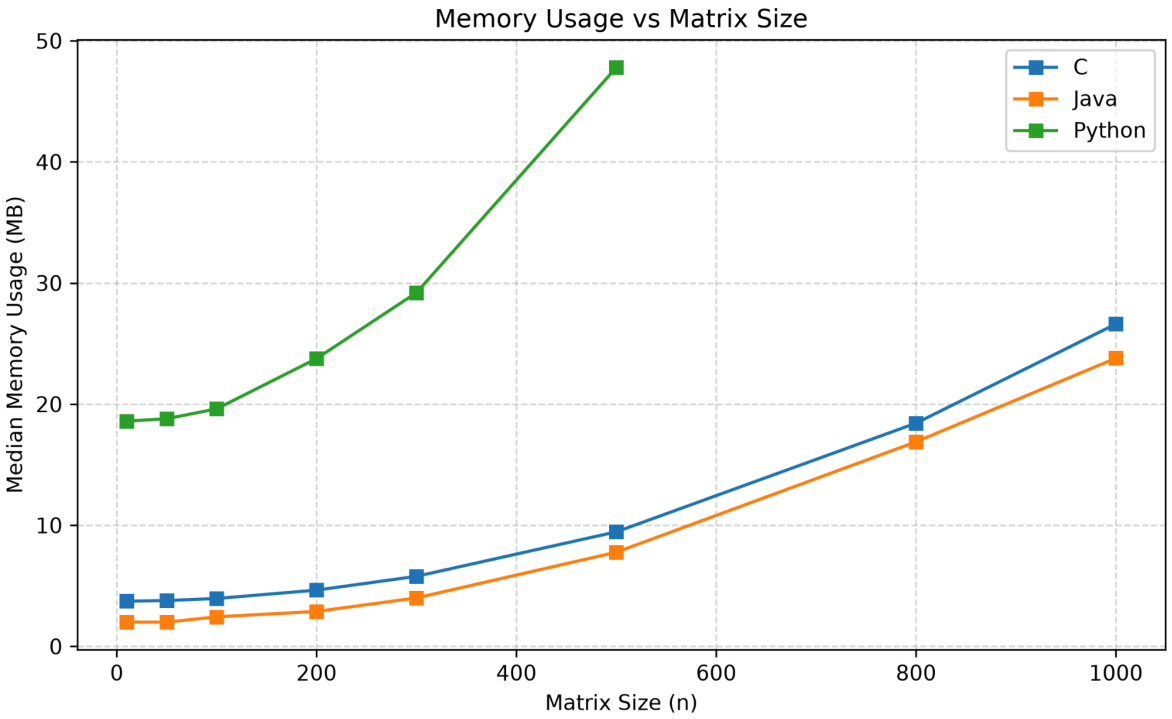


Figure 2: Median memory consumption (MB) vs. matrix size *n* across C (Peak Working Set), Java (Used Heap), and Python (Peak Working Set).

3.1 Summary of Runtime, Memory, and Execution Limits

Size (n)	C Median Time (s)	C Mem (MB)	Java Median Time (s)	Java Mem (MB)	Python Median Time (s)	Python Mem (MB)	Python Status
10	≈ 4.0×10^{-7}	≈ 3.7	≈ 1.4×10^{-5}	≈ 2.0	≈ 6.5×10^{-5}	≈ 18.6	COMPLETED (5/5)
50	≈ 3.5×10^{-5}	≈ 3.8	≈ 2.4×10^{-4}	≈ 2.1	≈ 6.8×10^{-3}	≈ 18.8	COMPLETED (5/5)
100	≈ 3.0×10^{-4}	≈ 3.9	≈ 6.0×10^{-4}	≈ 2.5	≈ 5.8×10^{-2}	≈ 19.6	COMPLETED (5/5)
200	≈ 3.1×10^{-3}	≈ 4.7	≈ 4.2×10^{-3}	≈ 3.0	≈ 5.0×10^{-1}	≈ 23.8	COMPLETED (5/5)
300	≈ 1.2×10^{-2}	≈ 5.8	≈ 1.9×10^{-2}	≈ 4.0	≈ 1.90	≈ 29.2	COMPLETED (5/5)
500	≈ 6.0×10^{-2}	≈ 9.5	≈ 1.2×10^{-1}	≈ 7.9	≈ 11.50	≈ 47.8	COMPLETED (5/5)
800	≈ 5.2×10^{-1}	≈ 18.4	≈ 5.5×10^{-1}	≈ 16.9	—	—	TIMEOUT (>120 s)
1000	≈ 6.0×10^{-1}	≈ 26.6	≈ 2.80	≈ 23.8	—	—	*SKIPPED*

Note: C and Java completed all 8 configurations (up to n = 1000) well within the 120 s budget with very small IQR variability (visible primarily for Java at n = 50 during initial JIT tier transitions). Values marked with ≈ correspond to Figure 1 and Figure 2 plot readings; exact decimal values and IQRs are stored in `data/summary_results.csv`.

4. Discussion

4.1 Runtime Scaling and Execution Models: In Figure 1, all three curves exhibit straight parallel slopes on log-log axes for $n \geq 100$, confirming the theoretical $O(n^3)$ complexity: increasing n by a factor of 10 increases execution time by roughly 10^3 = 1,000×. However, the vertical offset between curves highlights the impact of the execution model. C compiled with `gcc -O2` executes native machine instructions directly on unboxed IEEE 754 registers. Java starts slower at n = 10 and n = 50 (where minor IQR dispersion reflects JIT compilation activity), but once the HotSpot JIT compiler optimizes the hot inner loop, Java tracks remarkably close to C between n = 200 and n = 800. By contrast, CPython executes roughly 190× slower than C at n = 500 (11.5 s vs. 0.06 s) because every inner-loop iteration incurs bytecode interpretation, list indexing checks, and allocation/unboxing of immutable `PyFloatObject` instances. At n = 800, a

single Python multiplication requires ~47 s, so the 7 total invocations (2 warm-ups + 5 repetitions ≈ 330 s) exceed the 120 s timeout budget.

4.2 Memory Layout, Cache Effects, and Metric Limitations: Theoretically, three dense $n \times n$ matrices of 8-byte floats require $3 \times 8n^2$ bytes (24 MB decimal / 22.89 MiB at $n = 1000$). At $n = 1000$, Java's runtime curve jumps steeper than C's (2.80 s vs. 0.60 s). In the `(i, j, k)` loop order, matrix B is accessed column-wise (`B[k][j]`). In C, traversing a contiguous 1D array with stride n causes L1/L2 cache misses as n grows, but hardware prefetchers still operate over a single contiguous block. In Java, each step of `k` dereferences a separate row object pointer (`B[k]`), causing pointer-chasing and TLB/cache pressure once the 24 MB footprint exceeds the CPU last-level cache.

Regarding Figure 2, memory metrics must not be compared as identical quantities: C and Python report OS-level Peak Working Set (starting at ~3.7 MB for C and ~18.6 MB for the Python interpreter at $n = 10$), whereas Java reports used JVM heap (`totalMemory() - freeMemory()`), which starts near 2.0 MB and scales quadratically with $24n^2$ bytes, but can fluctuate if garbage collection runs between allocations. Furthermore, Python reaches 47.8 MB at $n = 500$ (where raw array data is only 3×2 MB = 6 MB) due to the 24-byte per-float object overhead and pointer arrays.

5. Conclusions

Within the scope of sequential, textbook `(i, j, k)` dense matrix multiplication on a single CPU core, this study establishes that while algorithmic growth remains strictly cubic ($O(n^3)$) across languages, runtime overhead and memory layout determine practical scalability limits. Under a 120-second budget, C and Java scaled to $n = 1000$ (~24–27 MB memory), whereas pure Python reached its feasible limit at $n = 500$ (11.5 s median time, 47.8 MB memory). How loop reordering (`i, k, j`), cache blocking, and sparse representations mitigate memory-hierarchy bottlenecks remains to be investigated in Assignment 2.

6. References and Reproducibility Materials

- **[1]** T. H. Cormen, C. E. Leiserson, R. L. Rivest, and C. Stein, *Introduction to Algorithms*, 4th ed. Cambridge, MA, USA: MIT Press, 2022, ch. 4, pp. 75–82.
- **[2]** R. E. Bryant and D. R. O'Hallaron, *Computer Systems: A Programmer's Perspective*, 3rd ed. Boston, MA, USA: Pearson, 2016, ch. 6 (The Memory Hierarchy), pp. 635–665.
- **[3]** D. Goldberg, "What every computer scientist should know about floating-point arithmetic," *ACM Computing Surveys*, vol. 23, no. 1, pp. 5–48, Mar. 1991.

Reproducibility Package & AI Disclosure: All source files (`src/matmul.c`, `src/MatMul.java`, `src/matmul.py`), raw measurements (`data/raw_measurements.csv`, `data/summary_results.csv`, `data/execution_limits.csv`), and the automated benchmark/plotting notebook are provided in the companion repository. Build commands: `gcc -O2 src/matmul.c -o src/matmul_c.exe -lpsapi` and `javac src/MatMul.java`.

AI Disclosure: Generative AI was used to assist in structuring and editing the English prose of this report; code execution, experimental measurements and result were verified by the author.